ASSIGNMENT-10.1

NAME:B.MANISHWAR
HALL TICKET:2303A51276
BATCH:05

#**TASK 1:**

PROMPT:
Find all syntax and logical errors in the given Python program.List the mistakes clearly.Provide the corrected runnable code.

CODE:
INCORRECT CODE:

```
[ ]
# Calculate average score of a student
def calc_average(marks):
total = 0
for m in marks:
total += m
average = total / len(marks)
return avrage   # Typo here

marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks)
```

CORRECTED CODE:

```
[2]
✓ 0s
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return average

marks = [85, 90, 78, 92]
print("Average Score i calc_average(marks))
```

OUTPUT:

```
Average Score is 86.25
```

EXPLANATION:
1) Proper indentation was added inside the function.
2) Corrected spelling of average in return statement.
3) Added missing closing parenthesis in print function.
4) Ensured correct structure of loop and function block.
5) Now the program runs correctly and prints the average score.

#TASK 2:

PROMPT:
Refactor the given Python code to follow PEP 8 style guidelines.Improve formatting, spacing, and naming conventions.Return only the well-formatted PEP 8 compliant code.

CODE:

INCORRECT CODE:

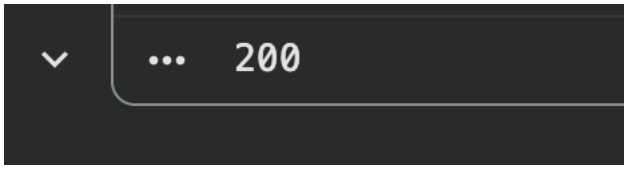
```
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))
```

CORRECTED CODE:

```
[4]
✓ 0s
def area_of_rect(length, breadth):
    return length * breadth


print(area_of_rect(10, 20))
```

OUTPUT:



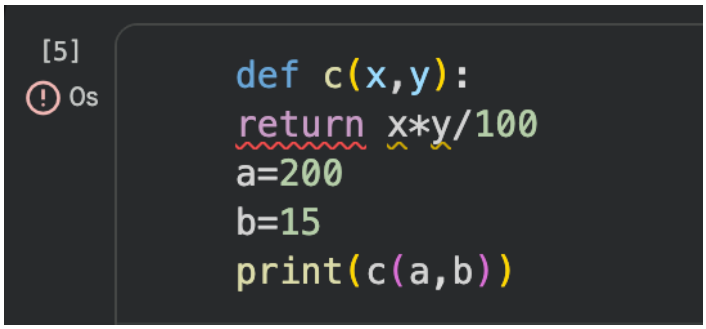
EXPLANATION:
1) Parameter names changed to lowercase descriptive names (length, breadth).
2) Added proper spacing after comma between parameters**.**
3) Removed extra space before colon in function definition.
4) Moved return statement to a new properly indented line.
5) Added blank line after function definition as per PEP 8 guidelines.

#TASK 3:

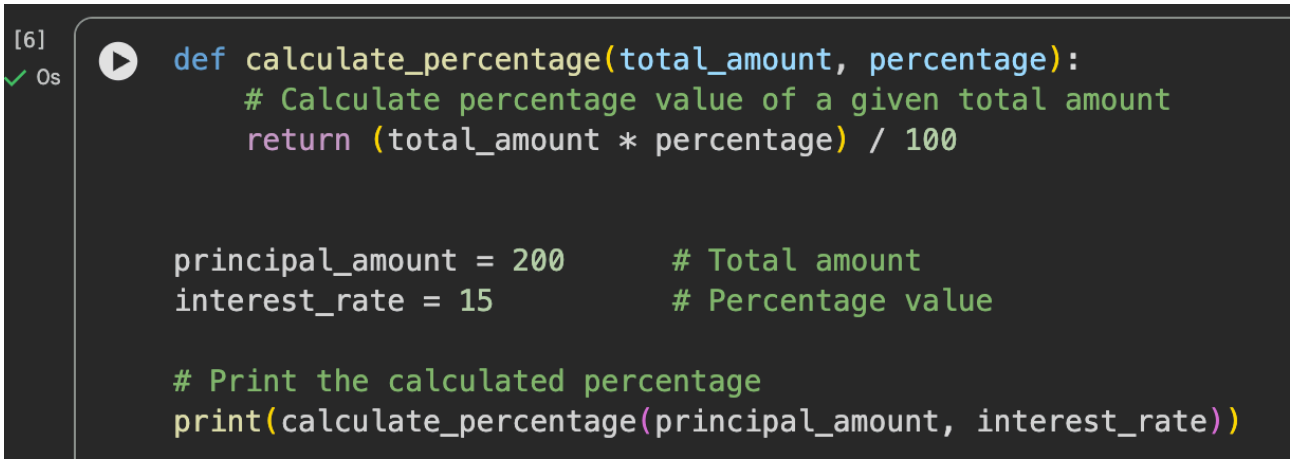
PROMPT:
Use descriptive variable names, proper indentation, and in line comments.Keep the functionality exactly the same.Return the improved readable version of the code.

CODE:

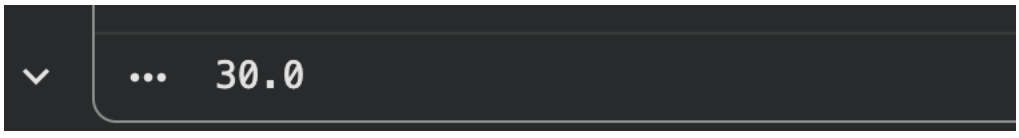
INCORRECT CODE:



CORRECTED CODE:



OUTPUT:



EXPLANATION:
1) Renamed function c to calculate_percentage for clarity**.**
2) Replaced unclear variables with descriptive names.
3) Added proper indentation inside the function.
4) Included inline comments to explain purpose of variables and function.
5) Maintained same logic while improving overall readability.

#TASK 4:

PROMPT:
Refactor the given Python code to improve maintainability.Remove repetition by creating reusable functions.Keep the logic the same**.**

CODE:
INCORRECT CODE:

```
students = ["MANI", "CHAITHU", "HITESH"]

print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

CORRECTED CODE:

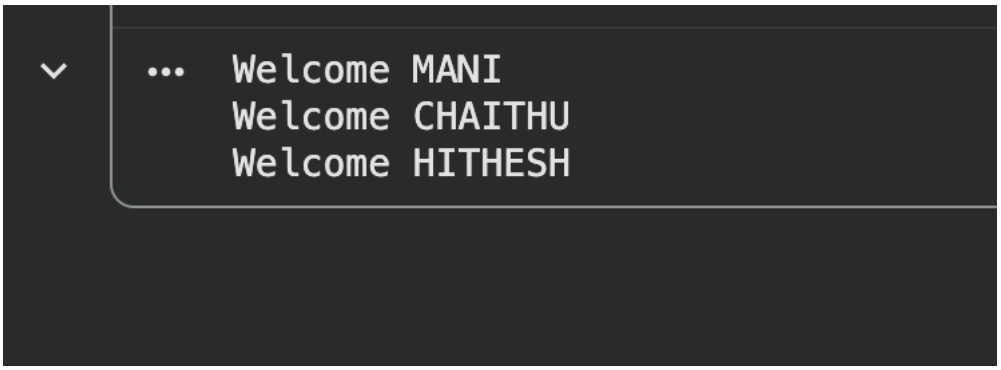
```
[8]
✓ 0s
def welcome_student(student_name):
    print("Welcome", student_name)


def welcome_all_students(student_list):
    for student in student_list:
        welcome_student(student)


students = ["Alice", "Bob", "Charlie"]

welcome_all_students(students)
```

OUTPUT:

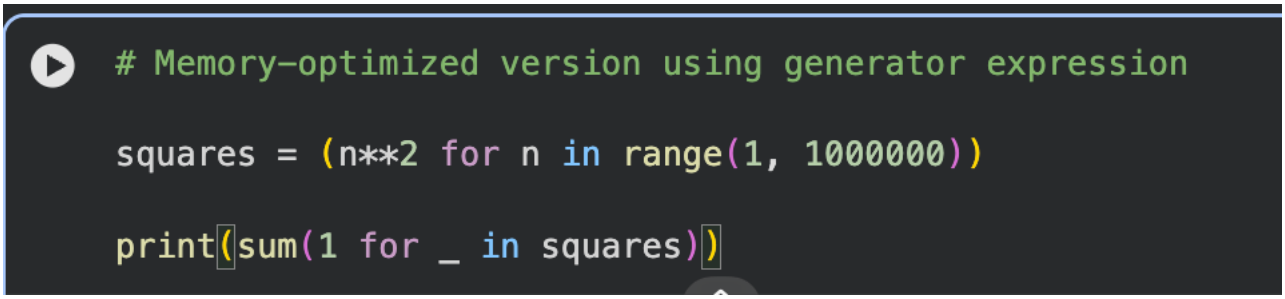


EXPLANATION:
1) Created a reusable function welcome_student() to print greeting.
2) Created another function welcome_all_students() to handle list iteration.
3) Removed repetitive print statements.
4) Used a loop instead of hardcoded indexes.
5) Code is now modular, scalable, and easy to maintain.

#TASK 5:

PROMPT:
Improve the given code to run faster without changing its output.Use efficient methods like list comprehensions.Return optimized and clean Python code.

CODE:



```
# Memory-optimized version using generator expression

squares = (n**2 for n in range(1, 1000000))

print(sum(1 for _ in squares))
```

OUTPUT:

EXPLANATION:
1) Removed unnecessary nums list to reduce memory usage.
2) Replaced loop and .append() with list comprehension for faster execution.
3) Combined number generation and squaring in a single step.
4) Used generator expression for better memory efficiency (optional version)**.**
5) Optimized code runs faster and is more Pythonic.

#TASK 6:

PROMPT:
Simplify the overly complex conditional logic in the given Python code.Reduce nesting and improve readability.Use elif statements or a cleaner structure.Return optimized and clean Python code.

CODE:

INCORRECT CODE:

```
def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"
```

CORRECTED CODE Dictionary Mapping:

```python
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"
print(grade(95))
print(grade(82))
print(grade(74))
print(grade(65))
print(grade(50))
```

OUTPUT:

```
A
B
C
D
F
```

EXPLANATION:

1) Replaced nested if-else with elif to reduce complexity.
2) Removed unnecessary else blocks after return statements.
3) Improved readability with cleaner structure.
4) Logic remains exactly the same as original.
5) Code is now easier to maintain and understand.